

Algorithm Selection Expert System

Comprehensive Technical Documentation

Language: SWI-Prolog | Interface: HTTP Web Server | Port: 8080

1. Project Overview

The Algorithm Selection Expert System is a web-based Prolog application that helps developers and students choose the most appropriate algorithm for a given problem. Instead of manually comparing dozens of algorithms, a user describes their problem using five parameters: problem type, data size, memory constraints, priority, and data characteristics. The Prolog inference engine reasons through a structured rule base to produce a ranked recommendation with full explanation.

The system is built entirely in SWI-Prolog using the built-in HTTP library (`library(http/thread_httpd)`), which allows Prolog to serve HTML pages over a network port without any external web framework. All pages, forms, and responses are generated directly from Prolog predicates using the `html_write` DCG (Definite Clause Grammar) system.

1.1 System Architecture

The system follows a knowledge-base architecture where facts, rules, and dynamic data are stored in separate files that are loaded (consulted) by the main web interface file at startup.

Component	File	Role
Static Knowledge Base	algorithms.pl	All 36 algorithm facts with complexity and description
Recommendation Rules	rules.pl	44 Prolog rules + 4 recursive predicates + helpers
Explanation Engine	complexity.pl	<code>use_case/2</code> and <code>avoid_when/2</code> facts for every algorithm
Dynamic KB	query_log.pl	Dynamic declarations for runtime logging and custom algorithms
Web Server	web_interface.pl	HTTP handlers, HTML generation, all 6 pages

1.2 How Prolog is Used

This project uses Prolog not as a scripting language but as a genuine inference engine. The five inputs from the user are treated as a query, and Prolog searches through the clause database using its built-in unification and backtracking mechanism to find the first rule that matches. That rule determines the recommendation. This is fundamentally different from a lookup table because multiple conditions must all be true simultaneously for a rule to fire.

1.3 Pages Served

URL Route	Page Name	Description
/	Home	Welcome page with live stats and category overview

URL Route	Page Name	Description
/select	Find Algorithm	Five-question input form using radio cards
/result	Recommendation	Full result: complexity, explanation, ranked comparison table
/browse	Browse All	Filterable card grid of all 36+ algorithms
/manage	Manage KB	Add / remove algorithms at runtime using assertz/retract
/history	History	Table of all past queries logged with bagof grouping

2. Prolog Fundamentals Used in This Project

This section explains every Prolog concept used in the project with definitions, purpose, and examples taken directly from the source code.

2.1 Facts

A fact is the simplest unit in a Prolog knowledge base. It is an unconditional statement that is always true. Facts are how static data is encoded in Prolog.

Syntax: `predicate_name(arg1, arg2, ..., argN).`

In this project, the `algorithm/6` predicate is the core fact. Each clause stores one algorithm:

```
algorithm(merge_sort, sorting, 'O(n log n)', 'O(n)', stable,
        'Divide-and-conquer sort. Guarantees O(n log n)').

% Fields: Name, Category, TimeComplexity, SpaceComplexity, Stability, Description
% Arity = 6, so this is algorithm/6
```

Other facts in the project:

- `use_case(Algorithm, Description)`: maps each algorithm to its real-world applications
- `avoid_when(Algorithm, Warning)`: maps each algorithm to conditions under which it fails
- `alternative(Category, Algo, AltAlgo)`: maps each recommended algorithm to a second option

2.2 Rules

A rule is a conditional statement. It says: "Head is true IF all goals in Body are true." Rules are how the expert system encodes decision logic.

Syntax: `Head :- Body.`

The primary rule in this project is `recommend/6`. Each clause handles one specific combination of constraints:

```
% Rule: if data is large AND stability is required THEN use merge_sort
recommend(sorting, large, _, stability, _, merge_sort) :- !.

% Rule: if memory is low THEN use heap_sort (only O(1)-space O(n log n) sort)
recommend(sorting, _, low, _, _, heap_sort) :- !.

% The underscore _ is an anonymous variable - it matches anything
% The cut ! prevents Prolog from backtracking past this clause
```

2.3 Unification

Unification is the process by which Prolog matches a query against the heads of clauses in the database. When a user selects "sorting, large, moderate, stability, random", Prolog tries to unify those values against the first argument positions of every recommend/6 clause until one matches completely.

```
% Query from the web form:
?- recommend(sorting, large, moderate, stability, random, Algo).

% Prolog tries each clause in order:
% Clause 1: recommend(sorting, small, _, simplicity, _, insertion_sort)
%   small /= large => FAIL, try next clause
% Clause 2: recommend(sorting, _, _, _, nearly_sorted, insertion_sort)
%   nearly_sorted /= random => FAIL, try next
% Clause 3: recommend(sorting, large, _, stability, _, merge_sort)
%   large=large, _=moderate, stability=stability, _=random => SUCCESS
%   Algo = merge_sort
```

2.4 The Cut (!)

The cut operator ! commits Prolog to the current clause and prevents it from backtracking to try alternative clauses. In this project, every recommend/6 rule ends with ! to ensure exactly one algorithm is recommended per query. Without cuts, Prolog might find multiple matching rules and produce multiple results.

```
recommend(sorting, _, low, _, _, heap_sort) :- !.
% The ! means: once this rule fires, stop. Do not try any other recommend clause.

% Without cut, if memory=low AND data=large, both of these could fire:
recommend(sorting, _, low, _, _, heap_sort) :- !.    % fires first
recommend(sorting, large, _, speed, _, quick_sort). % would also match
```

2.5 If-Then-Else (-> ;)

The if-then-else construct in Prolog is written as (Condition -> ThenGoal ; ElseGoal). It is used extensively in this project for conditional logic that must succeed even when the condition fails.

```
% In result_page: fetch use case or provide a default
( use_case(Algo, UseCase) -> true ; UseCase = 'No use case recorded.' )

% In score_algorithm: award points for matching constraints
( recommend(_, DS, Mem, Pri, DO, Algo) -> P1 = 30 ; P1 = 0 ),
( Pri = stability, Stab = stable -> P2 = 20 ; P2 = 0 ),

% In manage_page: validate form fields
( NameStr = "" ; CatStr = "" ; ... ) ->
  render_manage_page("error", "All fields are required.")
```

2.6 findall/3

`findall(Template, Goal, List)` collects all solutions to `Goal` into `List`, instantiating `Template` for each. It never fails. If `Goal` has no solutions, `List` is empty `[]`. This is used throughout the project to build lists from the knowledge base.

```
% Collect every algorithm name in a given category
all_in_category(Category, List) :-
    findall(Name, algorithm(Name, Category, _, _, _, _), List).

% Collect total count of all algorithms
findall(_, algorithm(_, _, _, _, _, _), AllA),
length(AllA, TotalAlgos),

% Collect category stats: each Cat paired with its count
findall(Cat-Count,
    ( member(Cat, Cats),
      findall(_, algorithm(_, Cat, _, _, _, _), Algos),
      length(Algos, Count) ),
    StatsList).
```

2.7 bagof/3

`bagof(Template, Goal, List)` is similar to `findall` but it groups solutions by free variables. If the `Goal` has no solutions, `bagof` fails (unlike `findall` which gives `[]`). Free variables not being grouped must be existentially quantified with the caret `^` operator.

```
% Collect all stable algorithm names using bagof
% Time^Space^Desc^ means: existentially quantify Time, Space, Desc
% (group only by Name and Cat, ignoring those three)
bagof(Name-Cat,
    Time^Space^Desc^algorithm(Name, Cat, Time, Space, stable, Desc),
    StableList)

% In history_page: group queries by problem type
bagof(PT, ID^DS^Mem^Pri^DO^Algo^query_log(ID,PT,DS,Mem,Pri,DO,Algo), PTList)
```

2.8 member/2

`member(Element, List)` succeeds if `Element` is a member of `List`. It can be used for checking membership (deterministic) or generating members (non-deterministic via backtracking).

```
% Check if space complexity is O(1) or O(log n)
member(Space, ['O(1)', 'O(log n)'])

% Check if data order matches a known high-priority characteristic
member(DO, [nearly_sorted, integer_small_range, float_uniform,
            unweighted, weighted_negative, knapsack, ...])

% In category_stats: iterate over categories
member(Cat, Cats)
```

2.9 forall/2

`forall(Condition, Action)` succeeds if `Action` is true for every solution of `Condition`. It is equivalent to saying "there is no case where `Condition` is true but `Action` is false."

```
% Verify that every algorithm in a given list exists in the KB
all_exist(AlgoList) :-
    forall(member(A, AlgoList), algorithm(A, _, _, _, _)).

% Reading: "For all A that are members of AlgoList,
%         algorithm(A,...) must succeed."
```

2.10 Negation-as-Failure (\+)

`\+ Goal` succeeds if `Goal` cannot be proven. It does NOT mean `Goal` is logically false; it means Prolog cannot find a proof. This is called the Closed World Assumption: anything not in the database is assumed false.

```
% Prevent adding a duplicate algorithm
add_algorithm(Name, Cat, Time, Space, Stab, Desc) :-
    \+ algorithm(Name, _, _, _, _), % fail if Name already exists
    assertz(algorithm(Name, Cat, Time, Space, Stab, Desc)),
    assertz(custom_algorithm(Name, Cat, Time, Space, Stab, Desc)).

% In rules.pl: filter algorithms without n^2 time complexity
fast_algorithms(Category, List) :-
    findall(Name,
        ( algorithm(Name, Category, Time, _, _, _),
          \+ sub_string(Time, _, _, _, "n^2") ),
        List).
```

2.11 assertz/1 and retract/1: Dynamic Knowledge Base

`assertz(Fact)` adds a new fact to the database at runtime, at the end of existing clauses. `retract(Fact)` removes the first matching fact. `retractall(Fact)` removes all matching facts. These predicates allow the knowledge base to be modified while the program is running, which is what makes it a dynamic knowledge base.

```
% Add a custom algorithm to the live KB
add_algorithm(Name, Cat, Time, Space, Stab, Desc) :-
    \+ algorithm(Name, _, _, _, _),
    assertz(algorithm(Name, Cat, Time, Space, Stab, Desc)),
    assertz(custom_algorithm(Name, Cat, Time, Space, Stab, Desc)).

% Remove a custom algorithm (only custom ones can be removed)
remove_algorithm(Name) :-
    custom_algorithm(Name, Cat, Time, Space, Stab, Desc),
    retract(algorithm(Name, Cat, Time, Space, Stab, Desc)),
    retract(custom_algorithm(Name, Cat, Time, Space, Stab, Desc)).

% Log every query to the dynamic database
save_query(ID, PT, DS, Mem, Pri, DO, Algo) :-
    assertz(query_log(ID, PT, DS, Mem, Pri, DO, Algo)).

% Clear all query history
```

```
clear_history :- retractall(query_log(_, _, _, _, _, _, _)).
```

2.12 DCG: Definite Clause Grammars (-->)

DCG rules use --> instead of :- and are used for generating structured output (in this case HTML). A DCG rule is automatically compiled by SWI-Prolog into a regular Prolog rule with two extra "difference list" arguments that thread an input and output list through the grammar. In this project, DCGs generate the HTML content that `reply_html_page` sends to the browser.

```
% DCG rule: render a category card
render_cat_cards([]) --> []. % base case
render_cat_cards([Cat-Count|Rest]) -->
    { fmt_atom(Cat, CatDisplay) }, % {} = regular Prolog goals
    html(div([class('cat-card')], [
        h3(CatDisplay),
        span([class('cat-count')], [Count, ' algorithms'])
    ]),
    render_cat_cards(Rest). % recursive call

% Called in HTML as: \render_cat_cards(Stats)
% The \ prefix tells html_write to call the DCG rule
```


3. File-by-File Code Explanation

3.1 algorithms.pl: The Static Knowledge Base

This file contains the core facts of the expert system. Every algorithm is represented as a single algorithm/6 fact. The predicate is declared dynamic at the top so that runtime-added algorithms can coexist with the static ones.

```
:- dynamic algorithm/6.  
% Without this declaration, assertz(algorithm(...)) would create  
% a separate dynamic predicate that could conflict with the static facts.
```

The algorithm/6 fact structure:

Argument	Position	Type	Example
Name	1st	Atom	merge_sort
Category	2nd	Atom	sorting
Time Complexity	3rd	Atom (string)	'O(n log n)'
Space Complexity	4th	Atom (string)	'O(n)'
Stability	5th	Atom	stable or unstable
Description	6th	Atom (string)	'Divide and conquer...'

Total: 11 sorting + 6 searching + 9 graph + 5 dynamic programming + 5 string matching = 36 built-in algorithms.

3.2 rules.pl: Recommendation Rules and Recursive Predicates

This is the most logically complex file. It contains three kinds of predicates: the recommend/6 rules that select algorithms, helper predicates using findall/bagof/member/forall, and four recursive predicates that build the ranked comparison table.

The recommend/6 Rule Structure

All 44 recommend rules follow a pattern of ordered specificity. More specific rules come first, with a fallback (default) rule at the end of each category. The cut ! after each rule body ensures that once a rule fires, no other rule is tried.

```
% SORTING RULES (11 rules + 1 fallback)  
  
% Rule 1: Tiny data - simplicity wins no matter what  
recommend(sorting, small, _, simplicity, _, insertion_sort) :- !.  
  
% Rule 2: Nearly sorted - insertion sort is provably best
```

```

recommend(sorting, _, _, _, nearly_sorted, insertion_sort) :- !.

% Rule 3: Large data + stability required = merge sort
recommend(sorting, large, _, stability, _, merge_sort) :- !.

% Rule 4: Medium data + stability = tim sort (best practical stable sort)
recommend(sorting, medium, _, stability, _, tim_sort) :- !.

% Rule 5: Memory tight = heap sort (only in-place guaranteed  $O(n \log n)$ )
recommend(sorting, _, low, _, _, heap_sort) :- !.

% Rule 6: Integers in small range = counting sort  $O(n+k)$ 
recommend(sorting, _, _, speed, integer_small_range, counting_sort) :- !.

% Rule 7: Integers in large range = radix sort (digit-by-digit)
recommend(sorting, _, _, speed, integer_large_range, radix_sort) :- !.

% Rule 8: Uniform floats = bucket sort
recommend(sorting, _, _, _, float_uniform, bucket_sort) :- !.

% Rule 9: Speed + large = quick sort
recommend(sorting, large, _, speed, _, quick_sort) :- !.

% Rule 10: Speed + medium = quick sort
recommend(sorting, medium, _, speed, _, quick_sort) :- !.

% Fallback: when nothing else matches
recommend(sorting, _, _, _, _, tim_sort).

```

Why the Order Matters

Prolog evaluates rules top-to-bottom. If "nearly sorted" were checked after "large data + stability", a user with large nearly-sorted data would get `merge_sort` instead of `insertion_sort` (which is optimal for nearly sorted data regardless of size). The ordering encodes expert knowledge about which conditions are most decisive.

Graph Rules: Correctness-Based Selection

Graph algorithm selection is particularly critical because choosing the wrong algorithm produces incorrect results, not just slow ones. The rules encode correctness constraints first:

```

% Negative weights: Dijkstra FAILS on these - Bellman-Ford is mandatory
recommend(graph, _, _, _, weighted_negative, bellman_ford) :- !.

% Unweighted graph: BFS gives optimal shortest path, Dijkstra is wasteful
recommend(graph, _, _, _, unweighted, bfs) :- !.

% Heuristic available: A* beats Dijkstra when  $h(n)$  is admissible
recommend(graph, _, _, _, heuristic, a_star) :- !.

% All-pairs on small dense graph: Floyd-Warshall is the only  $O(V^3)$  option
recommend(graph, small, _, _, all_pairs, floyd_warshall) :- !.

```

The Scoring System: score_algorithm/6

This predicate assigns a numeric fitness score to any algorithm against the current set of constraints. It is called by the recursive rank_algorithms pipeline to produce the comparison table.

```
score_algorithm(Algo, DS, Mem, Pri, DO, Score) :-
    algorithm(Algo, _, _, Space, Stab, _),
    % +30 if this is the primary recommendation for these constraints
    ( recommend(_, DS, Mem, Pri, DO, Algo) -> P1 = 30 ; P1 = 0 ),
    % +20 if stability is priority AND algorithm is stable
    ( Pri = stability, Stab = stable -> P2 = 20 ; P2 = 0 ),
    % +15 if memory is low AND algorithm uses O(1) or O(log n) space
    ( Mem = low, member(Space, ['O(1)', 'O(log n)']) -> P3 = 15 ; P3 = 0 ),
    % +10 if data characteristic is a known high-specificity condition
    ( member(DO, [nearly_sorted, integer_small_range, ...]) -> P4 = 10 ; P4 = 0 ),
    % +5 if dataset is large (rewards scalable algorithms)
    ( DS = large -> P5 = 5 ; P5 = 0 ),
    Score is P1 + P2 + P3 + P4 + P5.
```

3.3 Recursive Predicates in rules.pl

Four explicit recursive predicates are defined in rules.pl. Each follows the standard Prolog recursion pattern: a base case that handles the empty list, and a recursive case that processes the head of the list and recurses on the tail.

Recursion 1: score_all/6

Walks a list of algorithm names, calls score_algorithm on each, and builds a paired list of Score-AlgorithmName terms. This is the engine that powers the ranked comparison table.

```
% BASE CASE: empty list produces empty scored list
score_all([], _, _, _, _, []).

% RECURSIVE CASE: score the head, prepend to result, recurse on tail
score_all([Algo|Rest], DS, Mem, Pri, DO, [Score-Algo|ScoredRest]) :-
    score_algorithm(Algo, DS, Mem, Pri, DO, Score),
    score_all(Rest, DS, Mem, Pri, DO, ScoredRest).

% Called by rank_algorithms which sorts the result:
rank_algorithms(Category, DS, Mem, Pri, DO, Ranked) :-
    all_in_category(Category, Algos),
    score_all(Algos, DS, Mem, Pri, DO, Scored),
    msort(Scored, Ascending),
    reverse(Ascending, Ranked). % Best score first
```

Recursion 2: collect_warnings/2

Walks a list of algorithm names and collects avoid_when/2 warnings for each. Uses a cut in the first recursive case to avoid double-collecting if an algorithm has multiple warnings.

```
% BASE CASE
collect_warnings([], []).
```

```

% If head has a warning: prepend it and recurse
collect_warnings([Algo|Rest], [Algo-Warning|MoreWarnings]) :-
    avoid_when(Algo, Warning), !,
    collect_warnings(Rest, MoreWarnings).

% If head has no warning: skip it and recurse
collect_warnings([_|Rest], Warnings) :-
    collect_warnings(Rest, Warnings).

```

Recursion 3: filter_by_score/3

Walks a scored list (Score-Algo pairs) and keeps only entries whose score meets or exceeds a threshold. Used to filter out low-scoring algorithms before building the comparison table.

```

% BASE CASE: empty list
filter_by_score([], _, []).

% KEEP: score >= threshold, add to result, recurse
filter_by_score([Score-Algo|Rest], Threshold, [Score-Algo|Kept]) :-
    Score >= Threshold, !,
    filter_by_score(Rest, Threshold, Kept).

% SKIP: score < threshold, recurse without adding to result
filter_by_score([_|Rest], Threshold, Kept) :-
    filter_by_score(Rest, Threshold, Kept).

```

Recursion 4: build_comparison/3

Takes the filtered scored list and builds a list of algo/4 terms (structured compound terms) containing name, time complexity, space complexity, and stability for each entry. The Max parameter limits the table to 5 rows.

```

% BASE CASE 1: maximum rows reached
build_comparison(_, 0, []) :- !.

% BASE CASE 2: list exhausted
build_comparison([], _, []).

% RECURSIVE CASE: fetch details from KB, build term, decrement Max
build_comparison([_Score-Algo|Rest], Max, [algo(Algo,Time,Space,Stab)|More]) :-
    algorithm(Algo, _, Time, Space, Stab, _),
    Next is Max - 1,
    build_comparison(Rest, Next, More).

% The full pipeline in result_page:
rank_algorithms(PT, DS, Mem, Pri, DO, Ranked),      % score all
filter_by_score(Ranked, 5, TopCandidates),          % keep score >= 5
build_comparison(TopCandidates, 5, CompList),        % build max 5 rows

```

3.4 complexity.pl: The Explanation Engine

This file contains two types of supplementary facts that enrich the result page beyond the primary recommendation.

use_case/2 maps each algorithm to a real-world scenario where it is the natural choice:

```
use_case(merge_sort, 'External sorting of large disk files; sorting linked lists').
use_case(dijkstra,   'GPS navigation; network routing protocols; map
applications').
use_case(kmp,        'Text editors; DNA sequence search; single pattern matching').
```

avoid_when/2 encodes the conditions under which each algorithm should not be used. This is crucial expert knowledge that prevents common mistakes:

```
avoid_when(quick_sort, 'Nearly sorted data - degrades to  $O(n^2)$  without pivot
randomisation').
avoid_when(dijkstra,   'Graph has negative weight edges - produces incorrect
shortest paths').
avoid_when(memoization, 'Recursion depth is very large - risk of stack overflow
errors').
```

3.5 query_log.pl: Dynamic Declarations

This file is intentionally minimal. Its only purpose is to declare two dynamic predicates before any facts are asserted into them. This is required in SWI-Prolog when a predicate has no static clauses but will have facts added at runtime.

```
:- dynamic query_log/7.
% query_log(ID, ProblemType, DataSize, Memory, Priority, DataOrder, Algorithm)

:- dynamic custom_algorithm/6.
% custom_algorithm(Name, Category, Time, Space, Stable, Description)
% Mirrors algorithm/6 but only for user-added algorithms,
% allowing selective removal without deleting built-in facts.
```

3.6 web_interface.pl: The HTTP Server

This is the largest file (977 lines) and serves three roles: it starts the HTTP server, defines all page handler predicates, and contains the global CSS. It loads all other files at startup using the consult shorthand.

```
:- use_module(library(http/thread_httpd)). % multi-threaded HTTP server
:- use_module(library(http/http_dispatch)). % route URLs to handler predicates
:- use_module(library(http/http_parameters)). % parse GET/POST parameters
:- use_module(library(http/html_write)). % DCG-based HTML generation

:- [algorithms]. % load algorithms.pl
:- [rules]. % load rules.pl
:- [complexity]. % load complexity.pl
:- [query_log]. % load query_log.pl
```

Route Registration

Each `http_handler/3` directive maps a URL path to a Prolog predicate. When a browser sends a request to that URL, SWI-Prolog calls the corresponding predicate with the HTTP request object as its argument.

```
:- http_handler(root(.),      home_page,      []).
:- http_handler(root(select), select_page,    []).
:- http_handler(root(result), result_page,    []).
:- http_handler(root(browse), browse_page,    []).
:- http_handler(root(history), history_page,   []).
:- http_handler(root(manage), manage_page,     []).
:- http_handler(root(add_algo), add_algo_handler, []).
:- http_handler(root(del_algo), del_algo_handler, []).
:- http_handler(root(clear),  clear_page,     []).
```

The result_page Handler: Full Inference Pipeline

This is the most important page handler. It orchestrates the full expert system pipeline from parsing user input to rendering the result. It is shown here with inline explanation of every step:

```
result_page(Request) :-
    % Step 1: Extract the 5 parameters from the HTTP GET request
    http_parameters(Request, [
        problem_type(PT, [atom, default(sorting)]),
        data_size(DS, [atom, default(medium)]),
        memory(Mem, [atom, default(moderate)]),
        priority(Pri, [atom, default(speed)]),
        data_order(DO, [atom, default(random)])
    ]),

    % Step 2: Run the expert system rule engine
    ( recommend(PT, DS, Mem, Pri, DO, Algo) ->

        % Step 3: Log the query to the dynamic KB
        next_query_id(ID),
        save_query(ID, PT, DS, Mem, Pri, DO, Algo),

        % Step 4: Fetch algorithm details from KB
        algorithm(Algo, _, Time, Space, Stability, Desc),

        % Step 5: Fetch explanation facts (use-case, warning)
        ( use_case(Algo, UseCase) -> true ; UseCase = 'No use case recorded.' ),
        ( avoid_when(Algo, AvoidMsg) -> true ; AvoidMsg = 'No specific warnings.'
    ),

    % Step 6: Find alternative algorithm
    ( alternative(PT, Algo, AltAlgo) -> true ; AltAlgo = none ),

    % Step 7: Compute category stats using findall + bagof
    findall(N, algorithm(N, PT, _, _, _, _), AllInCat),
    length(AllInCat, CatCount),
    ( bagof(N, T^S^D^algorithm(N,PT,T,S,stable,D), StableList)
    -> length(StableList, StableCount) ; StableCount = 0 ),

    % Step 8: Build ranked comparison using recursive pipeline
    rank_algorithms(PT, DS, Mem, Pri, DO, Ranked),
    filter_by_score(Ranked, 5, TopCandidates),
    build_comparison(TopCandidates, 5, CompList),
```

```
    % Step 9: Render HTML response
    reply_html_page(title('Recommendation'), [...])
;
    % No rule matched: show error page
    reply_html_page(title('No Match'), [...])
).
```

4. Full Algorithm Reference

This section documents every algorithm in the knowledge base with its facts, complexity, use cases, and warnings.

4.1 Sorting Algorithms

Algorithm	Time	Space	Stable	Best Used When
Bubble Sort	$O(n^2)$	$O(1)$	Yes	Teaching; detecting near-sorted data
Selection Sort	$O(n^2)$	$O(1)$	No	Write operations are expensive
Insertion Sort	$O(n^2)$	$O(1)$	Yes	Small data or nearly sorted input
Merge Sort	$O(n \log n)$	$O(n)$	Yes	Large data + stability required
Quick Sort	$O(n \log n)$	$O(\log n)$	No	Speed priority, large random data
Heap Sort	$O(n \log n)$	$O(1)$	No	Memory is very tight
Counting Sort	$O(n+k)$	$O(k)$	Yes	Integer data with small known range
Radix Sort	$O(nk)$	$O(n+k)$	Yes	Fixed-length integers or strings
Tim Sort	$O(n \log n)$	$O(n)$	Yes	General use (Python/Java default)
Shell Sort	$O(n \log^2 n)$	$O(1)$	No	Embedded systems with moderate data
Bucket Sort	$O(n+k)$	$O(n)$	Yes	Uniformly distributed floats

4.2 Searching Algorithms

Algorithm	Time	Space	Requirement	Best Used When
Linear Search	$O(n)$	$O(1)$	None	Data is unsorted or small
Binary Search	$O(\log n)$	$O(1)$	Sorted	Large sorted arrays
Jump Search	$O(\sqrt{n})$	$O(1)$	Sorted	Sorted array, simpler than binary
Interpolation Search	$O(\log \log n)$	$O(1)$	Uniform+Sorted	Phone book style uniform data
Exponential Search	$O(\log n)$	$O(1)$	Sorted	Unbounded or infinite arrays
Ternary Search	$O(\log_3 n)$	$O(1)$	Unimodal	Finding max/min of unimodal function

4.3 Graph Algorithms

Algorithm	Time	Space	Best Used When
BFS	$O(V+E)$	$O(V)$	Shortest path in unweighted graph
DFS	$O(V+E)$	$O(V)$	Cycle detection, topological sort
Dijkstra	$O(E \log V)$	$O(V)$	Shortest path, non-negative weights
Bellman-Ford	$O(VE)$	$O(V)$	Negative weights, detect negative cycles
A*	$O(E)$	$O(V)$	Shortest path with a good heuristic
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-pairs shortest path, small dense graph
Kruskal	$O(E \log E)$	$O(V)$	Minimum spanning tree, sparse graph
Prim	$O(E \log V)$	$O(V)$	Minimum spanning tree, dense graph
Topological Sort	$O(V+E)$	$O(V)$	Ordering DAG: build systems, scheduling

4.4 Dynamic Programming

Algorithm	Time	Space	Best Used When
Memoization	$O(n)$	$O(n)$	Natural recursion; top-down DP
Tabulation	$O(n)$	$O(n)$	Bottom-up DP; speed priority; memory not tight
Knapsack DP	$O(n*W)$	$O(n*W)$	Optimise value under weight/budget constraint
LCS DP	$O(m*n)$	$O(m*n)$	Sequence alignment, diff tools, DNA analysis
LIS DP	$O(n \log n)$	$O(n)$	Longest increasing subsequence problems

4.5 String Matching Algorithms

Algorithm	Time	Space	Best Used When
Naive	$O(m*n)$	$O(1)$	Short texts or teaching
KMP	$O(m+n)$	$O(m)$	Single pattern, guaranteed linear time
Rabin-Karp	$O(m+n)$	$O(1)$	Multiple patterns in one scan
Boyer-Moore	$O(m*n)$	$O(m)$	Large text, large alphabet (e.g. grep)
Z-Algorithm	$O(m+n)$	$O(n)$	Alternative to KMP, competitive programming

5. Decision Logic: Worked Examples

This section traces how the expert system reasons through specific queries step by step, showing exactly which rule fires and why.

Example 1: Sorting, Large, Low Memory, Speed, Random

```
?- recommend(sorting, large, low, speed, random, Algo).

% Prolog tries each rule in order:
% Rule 1: recommend(sorting, small, ...) -> small /= large FAIL
% Rule 2: recommend(sorting, _, _, _, nearly_sorted, ...) -> random /=
nearly_sorted FAIL
% Rule 3: recommend(sorting, large, _, stability, ...) -> speed /= stability FAIL
% Rule 4: recommend(sorting, medium, _, stability, ...) -> large /= medium FAIL
% Rule 5: recommend(sorting, _, low, _, _, heap_sort)
%         _ matches large, low=low, _ matches speed, _ matches random
%         => SUCCESS, Algo = heap_sort, ! prevents further search

% Result: heap_sort
% Why: Memory is low, so in-place O(1) space is mandatory.
%       Heap sort is the only algorithm that is both O(n log n) and O(1) space.
```

Example 2: Graph, Medium, Moderate, Speed, Weighted_Negative

```
?- recommend(graph, medium, moderate, speed, weighted_negative, Algo).

% Rule: recommend(graph, _, _, _, unweighted, bfs)
%       weighted_negative /= unweighted FAIL
% Rule: recommend(graph, _, _, _, cycle_detection, dfs)
%       weighted_negative /= cycle_detection FAIL
% Rule: recommend(graph, _, _, _, topological, topological_sort)
%       weighted_negative /= topological FAIL
% Rule: recommend(graph, _, _, _, heuristic, a_star)
%       weighted_negative /= heuristic FAIL
% Rule: recommend(graph, _, _, _, weighted_negative, bellman_ford)
%       _ matches medium, _ matches moderate, _ matches speed,
%       weighted_negative = weighted_negative => SUCCESS

% Result: bellman_ford
% Why: Graph has negative weights. Dijkstra is mathematically incorrect
%       in this case. Bellman-Ford is the only valid choice.
```

Example 3: How the Scoring Pipeline Works

After the primary recommendation is made, the recursive pipeline ranks all algorithms in the same category. Here is how it works for sorting with (large, low, speed, random):

```
% Step 1: all_in_category(sorting, Algos)
% Algos = [bubble_sort, selection_sort, insertion_sort, merge_sort,
%         quick_sort, heap_sort, counting_sort, radix_sort,
%         tim_sort, shell_sort, bucket_sort]
```

```

% Step 2: score_all recursively scores each:
%   heap_sort:      P1=30 (recommend fires) + P3=15 (O(1) + low mem) = 45
%   quick_sort:     P1=0  + P5=5 (large data) = 5
%   merge_sort:     P1=0  + P5=5 = 5
%   bubble_sort:    P1=0   = 0
%   ... etc

% Step 3: msort then reverse = [45-heap_sort, 5-quick_sort, 5-merge_sort, ...]

% Step 4: filter_by_score(Ranked, 5, TopCandidates)
% Keeps: [45-heap_sort, 5-quick_sort, 5-merge_sort, ...] (all >= 5)

% Step 5: build_comparison(TopCandidates, 5, CompList)
% Produces: [algo(heap_sort,...), algo(quick_sort,...), algo(merge_sort,...),...]

```

6. The Web Interface: How SWI-Prolog Serves HTML

SWI-Prolog includes a built-in HTTP server library that allows Prolog to serve web pages without any external web framework. This section explains how the HTML generation system works.

6.1 How `html_write` Works

The `html_write` library provides a DCG-based HTML generation system. Instead of writing HTML as strings, you write Prolog terms that mirror the structure of HTML. The library converts these terms to properly escaped HTML strings.

```
% Prolog term:
div([class('hero')], [
    h1('Hello World'),
    p([class('subtitle')], 'This is Prolog serving HTML')
])

% Becomes:
% <div class="hero">
%   <h1>Hello World</h1>
%   <p class="subtitle">This is Prolog serving HTML</p>
% </div>

% The first argument is attributes, the second is content
% Content can be: atoms, numbers, or nested elements
```

6.2 `reply_html_page/2`

`reply_html_page(Title, Content)` generates a complete HTML page with `<html>`, `<head>`, and `<body>` tags and sends it as the HTTP response. Content is a list of DCG calls and HTML terms.

```
home_page(_Request) :-
    reply_html_page(title('Algorithm Selection Expert System'), [
        \page_style,          % calls the page_style DCG rule
        \navbar(home),       % calls navbar DCG rule with argument home
        div([class('hero')], [
            h1('Algorithm Selection Expert System'),
            ...
        ])
    ]).
```

6.3 DCG Rules for Reusable Components

Components shared across pages (navbar, radio cards, comparison rows) are written as DCG rules and called with the `\` prefix. The `{}` notation inside DCG rules wraps regular Prolog goals that compute values used in the HTML.

```
% Navbar DCG: renders the top navigation bar
```

```

navbar(Active) -->
  html(nav([class('navbar')], [
    \nav_link('/', 'Home', Active, home),
    \nav_link('/select', 'Find Algorithm', Active, select),
    \nav_link('/browse', 'Browse All', Active, browse),
    \nav_link('/manage', 'Manage KB', Active, manage),
    \nav_link('/history', 'History', Active, history)
  ])).

% nav_link: adds active class to the current page link
nav_link(Href, Label, Active, Key) -->
  { ( Active = Key -> Class = 'nav-btn active' ; Class = 'nav-btn' ) },
  html(a([href(Href), class(Class)], Label)).

```

6.4 The Manage KB Page: Dynamic KB in Action

This page demonstrates all dynamic KB operations. When a user submits the add form, the handler validates input, calls `add_algorithm` which uses `assertz` to insert two new facts, and then re-renders the page showing the updated count.

```

manage_page(Request) :-
  ( member(method(post), Request) ->
    % POST: form submitted - validate and add
    http_parameters(Request, [name(NameStr,...), ...]),
    ( ( NameStr = "" ; CatStr = "" ; ... ) ->
      render_manage_page("error", "All fields are required.")
    ;
      atom_string(NameAtom, NameStr),
      ( name_taken(NameAtom) ->
        render_manage_page("error", "Name already exists.")
      ;
        add_algorithm(NameAtom, CatAtom, ...),
        render_manage_page("success", "Algorithm added.")
      )
    )
  ;
  % GET: just show the form
  render_manage_page("", "")
).

```

7. Complete Data Flow

This section traces the full journey of a user query from browser to recommendation.

Step	Where	What Happens
1	Browser	User selects 5 options on /select form and clicks submit
2	HTTP GET	Browser sends GET /result?problem_type=sorting&data_size=large&memory=low&priority=speed&data_order=random
3	result_page/1	http_parameters/2 extracts and binds the 5 atoms from the URL
4	recommend/6	Prolog tries each rule top-to-bottom via unification and backtracking
5	Rule fires	First matching rule binds Algo and ! prevents further search
6	save_query/7	assertz adds query_log fact to dynamic DB with auto-incremented ID
7	algorithm/6	Facts lookup: bind Time, Space, Stability, Desc for the recommended Algo
8	use_case/2, avoid_when/2	Explanation facts fetched from complexity.pl
9	rank_algorithms/6	Recursive pipeline: score all, msort, reverse
10	filter_by_score/3	Recursive filter: keep score >= 5
11	build_comparison/3	Recursive builder: create up to 5 algo/4 terms
12	reply_html_page/2	html_write DCG generates HTML; HTTP library sends to browser
13	Browser	User sees recommendation, complexity cards, insights, comparison table

7.1 Knowledge Base Files: Load Order

```
swipl -s web_interface.pl

web_interface.pl loads:
  1. algorithms.pl    (36 algorithm/6 facts, :- dynamic algorithm/6)
  2. rules.pl        (44 recommend/6 rules + helper predicates)
  3. complexity.pl    (36 use_case/2 + 24 avoid_when/2 facts)
  4. query_log.pl     (:- dynamic query_log/7, :- dynamic custom_algorithm/6)

Then registers 9 HTTP handlers and calls start_server(8080).
SWI-Prolog starts a multi-threaded HTTP server on port 8080.
```

8. Requirements

Requirement	Status	Where	Key Predicate / Construct
Facts	Met	algorithms.pl, complexity.pl	algorithm/6, use_case/2, avoid_when/2
Rules	Met	rules.pl	recommend/6, 44 rules with cuts and conditions
Dynamic KB (add)	Met	web_interface.pl	assertz/1 in add_algorithm/6, save_query/7
Dynamic KB (remove)	Met	web_interface.pl	retract/1 in remove_algorithm/1
Dynamic KB (clear)	Met	web_interface.pl	retractall/1 in clear_history/0
Web Interface	Met	web_interface.pl	SWI-Prolog HTTP server, 6 pages
Recursion	Met	rules.pl	score_all/6, collect_warnings/2, filter_by_score/3, build_comparison/3
findall/3	Met	rules.pl, web_interface.pl	all_in_category/2, category_stats/1, custom_count/1
bagof/3	Met	rules.pl, web_interface.pl	all_stable_algorithms/1, history_page grouping
member/2	Met	rules.pl, web_interface.pl	all_inplace_algorithms/2, score_algorithm/6
forall/2	Met	rules.pl	all_exist/1
\+ negation	Met	rules.pl, web_interface.pl	add_algorithm/6 duplicate check, fast_algorithms/2
-> ; conditional	Met	rules.pl, web_interface.pl	44 rules + result_page + manage_page
! cut	Met	rules.pl	Every recommend/6 rule body
Backtracking	Met	rules.pl	recommend/6 multi-clause structure
aggregate_all/3	Met	web_interface.pl	next_query_id/1 for auto-increment

9. Running the Project

9.1 Prerequisites

- Install SWI-Prolog from <https://www.swi-prolog.org/Download.html>
- Ensure all five .pl files are in the same directory
- No additional libraries, frameworks, or databases required

9.2 Starting the Server

```
# Open terminal in the project directory
swipl -s web_interface.pl

# Expected output:
# =====
#   Algorithm Selection Expert System
#   CM2520 - Logic Programming
#   Server running on port 8080
#   Open: http://localhost:8080/
# =====

# Then open in browser:
# http://localhost:8080/
```

9.3 Testing the Expert System Directly in SWI-Prolog

You can also test predicates directly in the Prolog console after loading:

```
?- recommend(sorting, large, low, speed, random, Algo).
% Expected: Algo = heap_sort

?- recommend(graph, medium, high, speed, weighted_negative, Algo).
% Expected: Algo = bellman_ford

?- all_in_category(sorting, List), length(List, N).
% Expected: N = 11

?- rank_algorithms(sorting, large, low, speed, random, Ranked).
% Shows ranked list with scores

?- score_algorithm(heap_sort, large, low, speed, random, Score).
% Expected: Score = 50 (P1=30 + P3=15 + P5=5)
```

9.4 File Structure

File	Lines	Purpose
web_interface.pl	977	HTTP server, all pages, CSS

File	Lines	Purpose
rules.pl	375	Recommendation rules + recursion + helpers
algorithms.pl	74	Algorithm knowledge base (36 facts)
complexity.pl	91	Use cases and warnings (60 facts)
query_log.pl	20	Dynamic declarations